OpenAI Codex Subagents and Agent Customization

Research-backed implementation guide with configuration reference, architecture patterns, workflow design, and a four-agent document production pipeline

Prepared on 2026-04-03
Scope: Codex subagents, custom agents, AGENTS.md, skills, MCP, config layers, profiles, and practical rollout guidance

How this report was produced

This document follows the exact role split requested by the user: researcher (gather and structure the official material), writer (turn the research into a cohesive guide and produce the PDF), reviewer (check structure, readability, and visible rendering), and curator (compare this report against the earlier assistant answer and reconcile differences).

Executive summary

Codex subagents are specialized child agents that Codex spawns in parallel when you explicitly ask it to delegate work. They are intended to keep the main thread focused while exploration, review, testing, or log-heavy analysis runs in separate threads that later return distilled results.

A custom subagent is primarily defined as a standalone TOML file placed under ~/.codex/agents/ or .codex/agents/. For that standalone file format, only three fields are mandatory: name, description, and developer_instructions. Common optional fields include nickname_candidates, model, model_reasoning_effort, sandbox_mode, mcp_servers, and skills.config.

Subagents are only one layer in Codex customization. The broader stack is: AGENTS.md for persistent repo guidance, skills for reusable workflows, MCP for external tools and knowledge sources, and subagents for delegation. The most effective setups use them together rather than treating them as substitutes.

The official docs strongly suggest narrow and opinionated agents. The best patterns are read-heavy parallel specialists plus one implementation-focused writer, not many concurrent writers changing the same codebase at once.

Bottom-line recommendation

Start with one repository-level AGENTS.md, one or two skills for repeatable workflows, one docs or browser MCP integration if needed, and only then introduce a small set of custom subagents. Begin with max_depth = 1 and a single writer pattern. Do not over-parallelize code editing.

1. Sources and research method

This report is based primarily on the official OpenAI Codex documentation set current as of 2026-04-03, with emphasis on pages that directly govern setup, configuration, and operating semantics. The research intentionally prioritized the official documentation over third-party commentary because the request concerns implementation detail and current product behavior.

- Primary implementation pages: Subagents, Subagent concepts, Configuration Reference, Config Basics, Advanced Configuration, Sample Configuration, AGENTS.md guide, Agent Skills, MCP, Customization, and Best Practices.
- Secondary context pages: Codex Models, Codex CLI overview, and the Agents SDK integration page for Codex-as-an-MCP-server workflows.
- Validation method: cross-check the same setting or claim across setup, concept, and reference pages whenever a topic appears in more than one location.

2. The Codex customization stack

The official Customization page frames Codex as a layered system. In practice, this is the cleanest way to think about implementation and maintenance:

Layer	Purpose	Best use
AGENTS.md	Persistent guidance that travels with the repo	Rules, build/test commands, conventions, routing hints
Skills	Reusable workflows with instructions, scripts, references	Repeatable procedures such as releases, docs updates, commit hygiene
MCP	External tools and shared systems	Docs servers, browsers, issue trackers, design tools, internal knowledge
Subagents	Parallel delegation to specialized workers	Exploration, reviews, tests, triage, bounded implementation

These layers are complementary rather than competing. AGENTS.md shapes behavior, skills package repeatable processes, and MCP connects Codex to systems outside the local workspace. Subagents then let you delegate work to specialized workers that can inherit or override parts of that environment.

3. File types, storage locations, and discovery rules

Item	Canonical location(s)	Notes
User config	~/.codex/config.toml	User-level defaults for CLI and IDE extension
Project config	.codex/config.toml	Loaded only for trusted projects; nearer files override farther ones
Custom subagents	~/.codex/agents/ or .codex/agents/	One standalone TOML file per custom agent

Global instructions	~/.codex/AGENTS.md	Developer-wide working style and defaults
Repo instructions	AGENTS.md / AGENTS.override.md	Loaded along the path from project root to cwd
Repo skills	.agents/skills/	Scanned from current directory upward to the repo root
User skills	$HOME/.agents/skills	Reusable personal skills across repositories
Admin skills	/etc/codex/skills	Shared machine or container skills

Codex discovers project configuration by walking up from the working directory until it reaches a project root. By default a directory containing .git is treated as the project root, but project_root_markers can change that behavior. Setting project_root_markers = [] makes the current working directory act as the project root.

AGENTS.md is resolved by layer and proximity. Codex reads the global file in ~/.codex first, then walks from the project root down to the current directory and checks each directory in order for AGENTS.override.md, AGENTS.md, and then any configured fallback filenames. At most one instruction file is included per directory; a local override wins over a same-directory AGENTS.md.

```
# ~/.codex/config.toml
project_doc_fallback_filenames = ["TEAM_GUIDE.md", ".agents.md"]
project_doc_max_bytes = 65536
project_root_markers = [".git", ".hg", ".sl"]
```

4. Subagents: operating model and lifecycle

Codex enables subagent workflows in current releases, but it does not automatically spawn subagents just because a task is complex. The user or caller must explicitly ask for subagents or parallel agent work. When many subagents run, Codex orchestrates spawning, follow-up routing, waiting, and thread closure, then returns a consolidated result.

- Built-in agent types: default (general fallback), worker (execution-focused implementation agent), and explorer (read-heavy exploration agent).
- Subagent workflows consume more tokens than comparable single-agent runs because each worker uses its own model and tools.
- Use /agent in the CLI to inspect and switch between agent threads.
- Subagents inherit the current sandbox policy unless you override it, and live runtime overrides from the parent turn can be re-applied when a child is spawned.

Important approval nuance

Even if a custom agent file sets a default sandbox or approval posture, the parent turn's live runtime choices can still win. The docs explicitly call out interactive changes such as /approvals or --yolo as examples of parent runtime overrides that are re-applied to children.

5. Standalone custom agent schema

The standalone custom-agent TOML format is the primary documented authoring path. Each file defines exactly one custom agent and is loaded as a configuration layer for spawned sessions.

Field	Type	Required	Purpose
name	string	Yes	Stable identifier Codex uses when spawning or referring to the agent
description	string	Yes	Human-facing guidance for when the agent should be selected
developer_instructions	string	Yes	Core behavioral instructions for the agent
nickname_candidates	string[]	No	Optional pool of UI display nicknames

If optional agent fields are omitted, they inherit from the parent session. The docs explicitly name model, model_reasoning_effort, sandbox_mode, mcp_servers, and skills.config as inheritable examples.

```
name = "reviewer"
description = "PR reviewer focused on correctness, security, and missing tests."
developer_instructions = """
Review code like an owner.
Prioritize correctness, security, behavior regressions, and missing test coverage.
"""
nickname_candidates = ["Atlas", "Delta", "Echo"]
```

Nicknames are presentation-only. Codex still identifies the agent by name, not by nickname. Candidate nicknames must be unique, non-empty, and composed of ASCII letters, digits, spaces, hyphens, or underscores.

6. All relevant configuration parameters for creating and operating agents

6.1 Global subagent controls in [agents]

Key	Type	Default	Meaning
agents.max_threads	number	6	Maximum number of concurrently open agent threads
agents.max_depth	number	1	Maximum spawn nesting depth; root session starts at depth 0
agents.job_max_runtime_seconds	number	1800 per worker if unset in spawn_agents_on_csv	Default timeout for CSV fan-out workers

The docs recommend keeping max_depth at 1 unless you explicitly need recursive delegation. Increasing it can create repeated fan-out, higher token consumption, greater latency, and less predictable local resource use.

6.2 Role registration keys in config.toml

Key	Type	Meaning
agents.<name>.description	string	Role guidance shown when Codex chooses or spawns that role
agents.<name>.config_file	path string	Path to a TOML config layer for that role; relative to the config file that declares it
agents.<name>.nickname_candidates	array<string>	Display nickname pool for that role

```
[agents]
max_threads = 6
max_depth = 1

[agents.reviewer]
description = "Find correctness, security, and test risks in code."
config_file = "./agents/reviewer.toml"
nickname_candidates = ["Athena", "Ada"]
```

6.3 Model and reasoning keys

Key	Allowed values	Notes
model	string	Recommended starting point in Codex is gpt-5.4 for most agents
model_reasoning_effort	minimal | low | medium | high | xhigh	xhigh is model-dependent; high is useful for reviewer/security agents
model_reasoning_summary	auto | concise | detailed | none	Controls reasoning summary detail or disables summaries
model_verbosity	low | medium | high	Optional GPT-5 Responses API verbosity override

The Subagent Concepts page gives concrete role guidance: gpt-5.4 is the default starting point for most agents, gpt-5.4-mini is the preferred faster and cheaper option for lighter subagent work, and gpt-5.3-codex-spark remains a research-preview option for near-instant text-only iteration for ChatGPT Pro users. If you do not pin model or model_reasoning_effort, Codex can choose a setup automatically.

6.4 Sandbox and approval keys

Key	Allowed values / type	Meaning
sandbox_mode	read-only | workspace-write | danger-full-access	Filesystem and network policy during command execution
approval_policy	untrusted | on-request | never | granular object	When and how approvals are surfaced

sandbox_workspace_write.network_access	boolean	Outbound network access when sandbox_mode = workspace-write
sandbox_workspace_write.writable_roots	array<string>	Additional writable roots in workspace-write mode
sandbox_workspace_write.exclude_slash_tmp	boolean	Exclude /tmp from writable roots
sandbox_workspace_write.exclude_tmpdir_env_var	boolean	Exclude $TMPDIR from writable roots

The granular approval object can independently control sandbox_approval, rules, mcp_elicitations, request_permissions, and skill_approval. This matters in advanced environments where you want a narrower approval surface than the three simple modes.

6.5 Skills-related keys

Key	Type	Meaning
skills.config	array<object>	Per-skill enablement overrides
skills.config.<index>.path	path string	Path to a skill folder containing SKILL.md
skills.config.<index>.enabled	boolean	Enable or disable the referenced skill

```
[[skills.config]]
path = "/path/to/skill/SKILL.md"
enabled = false
```

6.6 MCP-related keys most relevant to agent design

Group	Keys
STDIO transport	command (required), args, env, env_vars, cwd
HTTP transport	url (required), bearer_token_env_var, http_headers, env_http_headers
Cross-cutting	enabled, required, enabled_tools, disabled_tools, scopes, oauth_resource, startup_timeout_sec / ms, tool_timeout_sec
OAuth callback	mcp_oauth_callback_port, mcp_oauth_callback_url, mcp_oauth_credentials_store

Codex supports both STDIO and streamable HTTP MCP servers. For project-scoped MCP configuration, the project must be trusted. The CLI and IDE extension share the same MCP configuration.

6.7 Web search and tool configuration related to agent workflows

Key	Allowed values	Meaning
web_search	cached | live | disabled	Top-level web search mode
tools.web_search	boolean or object	Optional per-tool config; object supports context_size,

		allowed_domains, and approximate user location
features.multi_agent	boolean	Stable feature flag for multi-agent collaboration tools; on by default
tools.view_image	boolean	Enable the local-image attachment tool view_image

By default, local tasks use cached web search results from an OpenAI-maintained index. In full-access sandbox settings such as --yolo, live results become the default. Even cached web search results should still be treated as untrusted content.

6.8 Related configuration that often matters in real deployments

- profiles and profile for named presets; profiles are currently experimental and not supported in the IDE extension.
- model_provider and model_providers.<id> for custom backends or proxies.
- shell_environment_policy for environment inheritance, allow-lists, deny-lists, and explicit overrides.
- sqlite_home for SQLite-backed agent job state and resumable runtime data.
- project trust and project_root_markers, which decide whether project-scoped .codex layers are read at all.

7. Skills: structure, metadata, and when to use them instead of subagents

Skills are usually the best fit for reusable workflows because they can package instructions, scripts, references, and assets while staying discoverable via metadata. Codex uses progressive disclosure for skills: it starts with metadata and loads the full SKILL.md only when it decides to use the skill.

```
my-skill/
├─ SKILL.md          # required
├─ scripts/          # optional executable helpers
├─ references/       # optional docs or examples
├─ assets/           # optional templates/resources
└─ agents/
   └─ openai.yaml    # optional appearance, policy, dependencies
```

A minimal SKILL.md uses YAML frontmatter with name and description, followed by instructions. Skills can be invoked explicitly with /skills or $skill-name, or implicitly when the description matches the task. Because implicit matching depends on description quality, descriptions should define both trigger conditions and boundaries.

```
---
name: skill-name
description: Explain exactly when this skill should and should not trigger.
---

Skill instructions for Codex to follow.
```

Optional skill metadata lives in agents/openai.yaml. It can define interface metadata for the app UI, policy.allow_implicit_invocation, and dependencies.tools entries that declare MCP dependencies. This is the cleanest way to package workflows that require external systems.

```
interface:
  display_name: "Optional user-facing name"
  short_description: "Optional user-facing description"
  icon_small: "./assets/small-logo.svg"
  icon_large: "./assets/large-logo.png"
  brand_color: "#3B82F6"
  default_prompt: "Optional surrounding prompt"
policy:
  allow_implicit_invocation: false
dependencies:
  tools:
    - type: "mcp"
      value: "openaiDeveloperDocs"
      description: "OpenAI Docs MCP server"
      transport: "streamable_http"
      url: "https://developers.openai.com/mcp"
```

8. AGENTS.md: what belongs there and what does not

AGENTS.md provides durable project guidance that is loaded before work begins. The official docs explicitly say to keep it small. It is best for instructions that should apply every time in a repository or subtree, such as build commands, test commands, review expectations, conventions, and routing hints.

- Good AGENTS.md content: build and test commands, lint expectations, codebase conventions, and directory-specific routing guidance.
- Poor AGENTS.md content: very long procedure manuals, tool-specific workflows that belong in skills, or transient one-off instructions.
- Feedback loop pattern: when Codex makes the same mistake repeatedly, codify the fix in AGENTS.md so future sessions inherit the correction.

```
# ~/.codex/AGENTS.md
## Working agreements
- Always run npm test after modifying JavaScript files.
- Prefer pnpm when installing dependencies.
- Ask for confirmation before adding new production dependencies.

# repo-root/AGENTS.md
## Repository expectations
- Run npm run lint before opening a pull request.
- Document public utilities in docs/ when behavior changes.
```

9. Step-by-step implementation guide for custom agents in Codex

Step 1 — Decide the split between AGENTS.md, skills, MCP, and subagents

1. Write down your persistent repository rules. Put only those in AGENTS.md.
2. List repeatable workflows. Turn those into skills if they need richer instructions, references, or scripts.
3. List external systems the agent must query or operate. Connect those through MCP.
4. Only then define subagents for bounded delegation roles such as explorer, reviewer, docs researcher, browser debugger, or focused fixer.

Step 2 — Create or validate your baseline config

```
# ~/.codex/config.toml
model = "gpt-5.4"
approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "cached"

[agents]
max_threads = 6
max_depth = 1
```

For project-specific behavior, add .codex/config.toml inside the repository. Remember that project-scoped config only loads when the project is trusted.

Step 3 — Create the custom agent directory and files

```
mkdir -p .codex/agents
# Then create one TOML file per custom agent
```

5. Create .codex/agents/ for repo-specific agents or ~/.codex/agents/ for personal agents.
6. Add one TOML file per agent.
7. Use a filename that matches the name field when possible; the docs call this the simplest convention.
8. Keep each agent narrow and opinionated. Avoid broad “do everything” agents.

Step 4 — Start with a minimal agent, then add only the necessary overrides

```
name = "docs_researcher"
description = "Documentation specialist that verifies APIs and framework behavior."
developer_instructions = """
Use docs and exact references whenever possible.
Do not make code changes.
"""
```

If the role truly needs a different model, sandbox, MCP server, or skill override, add those explicitly. Otherwise let the agent inherit the parent session defaults.

Step 5 — Add MCP only when the agent needs external systems

```
[mcp_servers.openaiDeveloperDocs]
url = "https://developers.openai.com/mcp"
startup_timeout_sec = 20
```

Use STDIO when the server runs locally from a command. Use HTTP when the server is exposed by URL. Restrict tool exposure with enabled_tools or disabled_tools where appropriate.

Step 6 — Add skills if the role needs a reusable workflow

```
[[skills.config]]
path = "/absolute/path/to/skills/release-check/SKILL.md"
enabled = true
```

Skills are generally a better place than developer_instructions for long procedures, examples, helper scripts, or reference bundles.

Step 7 — Invoke subagents explicitly in your prompt

```
Review this branch with parallel subagents.
Spawn one subagent for security risks, one for test gaps, and one for maintainability.
```

```
Wait for all three, then summarize the findings by category with file references.
```

A strong subagent prompt states how work is divided, whether Codex should wait for all workers, and what output shape should be returned.

Step 8 — Validate behavior in the CLI or app

- Use /agent in the CLI to inspect spawned threads.
- Use /mcp to inspect active MCP servers.
- Ask Codex to summarize the active instructions to verify AGENTS.md layering.
- If a skill or config update does not appear, restart Codex.

Step 9 — Add profiles or one-off overrides only after the base design works

```
[profiles.deep-review]
model = "gpt-5.4"
model_reasoning_effort = "high"
approval_policy = "on-request"

# one-off
codex --config model=""gpt-5.4-mini""
codex --config sandbox_workspace_write.network_access=true
```

Profiles are useful for repeatable operating modes, but they are experimental and are not currently supported in the IDE extension. Use dedicated flags like --model when available, and use --config only for arbitrary keys.

Step 10 — Scale only after observability and behavior are stable

- Keep max_depth at 1 first.
- Expand thread count only if workers are truly independent.
- Prefer many readers and one writer.
- Use CSV fan-out only for repeated, row-shaped audits or summaries, not for arbitrary recursive orchestration.

10. Design patterns, defaults, and anti-patterns

Pattern	Why it works	Typical config
Many readers, one writer	Keeps investigation parallel while avoiding conflicting edits	explorer/reviewer/docs agents in read-only; one worker or fixer writes
Narrow and opinionated agents	Prevents role drift and keeps prompts predictable	Clear description, constrained developer_instructions, selective MCP/skills
Docs specialist via MCP	Moves API confirmation out of the main agent	read-only + docs MCP + no-code-change instructions
Browser debugger before UI fix	Separates reproduction evidence from implementation	browser_debugger with high reasoning and browser MCP, then ui_fixer

Anti-patterns to avoid

Do not create a single mega-agent that explores, reproduces, edits, reviews, and deploys. Do not let

multiple writer agents edit overlapping code paths in parallel. Do not hide long procedures inside developer_instructions when they should be packaged as skills. Do not rely on project-scoped config in untrusted repos.

11. Experimental CSV batch fan-out

The Subagents guide documents an experimental workflow called spawn_agents_on_csv. It is designed for repeated row-shaped work, not for general recursive orchestration. Codex reads a CSV, spawns one worker per row, waits for completion, and exports combined results to CSV.

- Useful for: one file/package/service per row; repeated incident or PR audits; structured summaries for many similar inputs.
- Inputs include csv_path, instruction with {column_name} placeholders, optional id_column, output_schema, output_csv_path, max_concurrency, and max_runtime_seconds.
- Each worker must call report_agent_job_result exactly once; otherwise the row is marked with an error in the exported CSV.

```
Create /tmp/components.csv with columns path,owner and one row per frontend component.
Then call spawn_agents_on_csv with:
- csv_path: /tmp/components.csv
- id_column: path
- instruction: "Review {path} owned by {owner}. Return JSON with keys path, risk, summary, and follow_up via
report_agent_job_result."
- output_csv_path: /tmp/components-review.csv
- output_schema: an object with required string fields path, risk, summary, and follow_up
```

12. Practical agent set requested by the user: researcher, writer, reviewer, curator

Below is a concrete four-agent design aligned with the user's requested pipeline for producing high-quality researched documentation. This is not a built-in OpenAI example; it is a recommended implementation synthesized from the official agent, skills, MCP, and workflow guidance.

12.1 researcher.toml

```
name = "researcher"
description = "Research specialist that gathers authoritative material, extracts implementation details, and returns
structured notes with explicit references."
model = "gpt-5.4"
model_reasoning_effort = "high"
sandbox_mode = "read-only"
developer_instructions = """
Understand the feature deeply before drafting conclusions.
Prefer official documentation, then corroborate with other high-quality sources when the topic benefits from
broader context.
Return structured notes organized as basics, required fields, optional fields, constraints, workflows, and gotchas.
Always include exact references or links when available.
Do not write final prose for publication quality; focus on evidence and structure.
"""
[mcp_servers.openaiDeveloperDocs]
url = "https://developers.openai.com/mcp"
```

Why this design: the researcher is the highest-context worker and benefits from stronger reasoning. It should remain read-only and evidence-oriented.

12.2 writer.toml

```
name = "writer"
description = "Documentation writer that turns research notes into a structured guide and prepares publishable document artifacts."
model = "gpt-5.4"
model_reasoning_effort = "medium"
sandbox_mode = "workspace-write"
developer_instructions = """
Write a coherent document from the approved research notes.
Preserve technical accuracy while improving hierarchy, structure, flow, and readability.
Organize the report into executive summary, implementation steps, configuration reference, examples, and references.
If generating a document artifact, make the smallest layout changes needed to improve readability without dropping content.
"""
```

Why this design: the writer may need workspace-write to create files, but it should not re-run broad research. Its job is composition, structure, and artifact production.

12.3 reviewer.toml

```
name = "reviewer"
description = "Quality reviewer for document structure, factual consistency, formatting defects, and visible rendering issues."
model = "gpt-5.4"
model_reasoning_effort = "high"
sandbox_mode = "read-only"
developer_instructions = """
Review the produced document like an owner.
Check for factual inconsistencies, omissions, broken hierarchy, awkward layout, visual defects, and citation gaps.
When reviewing a PDF, inspect the rendered output rather than trusting source text alone.
Return prioritized issues and concrete corrections.
Do not rewrite the entire document unless asked.
"""
```

Why this design: the reviewer must be independent from the writer and should focus on defects, not on re-authoring.

12.4 curator.toml

```
name = "curator"
description = "Reconciliation specialist that compares outputs, detects deltas, and produces a precise change log between versions or sources."
model = "gpt-5.4-mini"
model_reasoning_effort = "medium"
sandbox_mode = "read-only"
developer_instructions = """
Compare two artifacts or texts and explain what changed, what expanded, and whether any contradictions exist.
Focus on deltas, omissions, and reframing.
Return a concise but exact reconciliation note.
"""
```

Why this design: the curator is a comparison and reconciliation worker. It usually does not need heavy tool use or write access.

12.5 Orchestrator prompt example

```
Create a complete implementation guide for Codex subagents.
- Have researcher gather and structure the authoritative material, including required fields, optional fields, workflows, and gotchas.
- Have writer turn that material into a polished document and generate the PDF.
- Have reviewer inspect the rendered PDF for factual or visual issues and report corrections.
- Have curator compare the final PDF content against the earlier summary answer and list the differences.
Wait for all workers where appropriate and return a consolidated final report with links to the artifacts.
```

13. Curator comparison: this report versus the earlier assistant answer

Comparison result: no material contradiction was found between the earlier answer and this report. The earlier answer correctly identified the core standalone custom-agent schema, the main optional overrides, the key storage locations, and the recommended narrow-agent design pattern. This report is broader and more implementation-complete.

Area	Earlier answer	This report
Core standalone custom-agent schema	Covered	Covered again with tables and examples
[agents] global controls	Covered	Expanded with defaults, warnings, and CSV runtime notes
[agents.<name>] role registration in config.toml	Briefly noted	Documented explicitly with sample config
AGENTS.md resolution order and fallback filenames	Not detailed	Explained in depth
Skills structure and agents/openai.yaml metadata	Mentioned at a high level	Expanded with full folder model and metadata examples
MCP key surface	Summarized	Expanded with STDIO/HTTP required and optional keys
Profiles, one-off --config overrides, project_root_markers	Not covered or only lightly implied	Explicitly documented
tools.web_search object form and location options	Not detailed	Documented
Requested four-agent documentation pipeline	Not provided	Provided as concrete TOML templates
PDF review and output reconciliation workflow	Not applicable	Integrated as requested

In other words, the earlier answer was directionally correct but incomplete relative to the current request. This report adds implementation detail, configuration breadth, comparative role design, and artifact-oriented workflow guidance.

14. References

[R1] OpenAI Developers — Subagents. https://developers.openai.com/codex/subagents

[R2] OpenAI Developers — Subagent Concepts. https://developers.openai.com/codex/concepts/subagents

[R3] OpenAI Developers — Configuration Reference. https://developers.openai.com/codex/config-reference

[R4] OpenAI Developers — Config Basics. https://developers.openai.com/codex/config-basic

[R5] OpenAI Developers — Advanced Configuration. https://developers.openai.com/codex/config-advanced

[R6] OpenAI Developers — Sample Configuration. https://developers.openai.com/codex/config-sample

[R7] OpenAI Developers — Custom instructions with AGENTS.md. https://developers.openai.com/codex/guides/agents-md

[R8] OpenAI Developers — Agent Skills. https://developers.openai.com/codex/skills

[R9] OpenAI Developers — Customization. https://developers.openai.com/codex/concepts/customization

[R10] OpenAI Developers — Model Context Protocol. https://developers.openai.com/codex/mcp

[R11] OpenAI Developers — Codex Models. https://developers.openai.com/codex/models

[R12] OpenAI Developers — Best practices. https://developers.openai.com/codex/learn/best-practices

[R13] OpenAI Developers — Use Codex with the Agents SDK. https://developers.openai.com/codex/guides/agents-sdk

Appendix A. Minimal practical starter pack

```
# .codex/config.toml
model = "gpt-5.4"
approval_policy = "on-request"
sandbox_mode = "workspace-write"
web_search = "cached"

[agents]
max_threads = 6
max_depth = 1

# .codex/agents/reviewer.toml
name = "reviewer"
description = "PR reviewer focused on correctness, security, and missing tests."
model = "gpt-5.4"
model_reasoning_effort = "high"
sandbox_mode = "read-only"
developer_instructions = """
Review code like an owner.
Prioritize correctness, security, regressions, and missing test coverage.
"""
```

This starter pack is intentionally small. It gives you a safe baseline before you add MCP, skills, or more specialized workers.